

UNIVERSITY OF THESSALY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

ECE433 - COMPUTER GRAPHICS WITH OpenGL

Project 1: Drawing Lines and Curves

Group members

Ιωάννης Ιάσων ΝΙΚΑΣ - 3771

Αναστάσιος ΚΑΛΟΤΣΗΣ - 3792

Παναγιώτης Νικόλαος ΤΣΟΓΚΑΣ -
3672

Group number: 01

July 20, 2026 (Fall 2025)

https://eclass.uth.gr/courses/E-CE_U_244/

Exercise 1

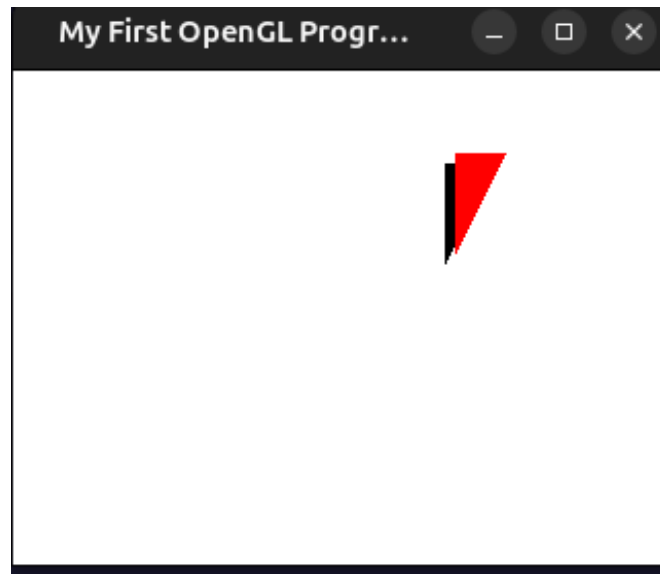


Figure 1: Αλλάξαμε το window size από 640x480 σε 320x240.

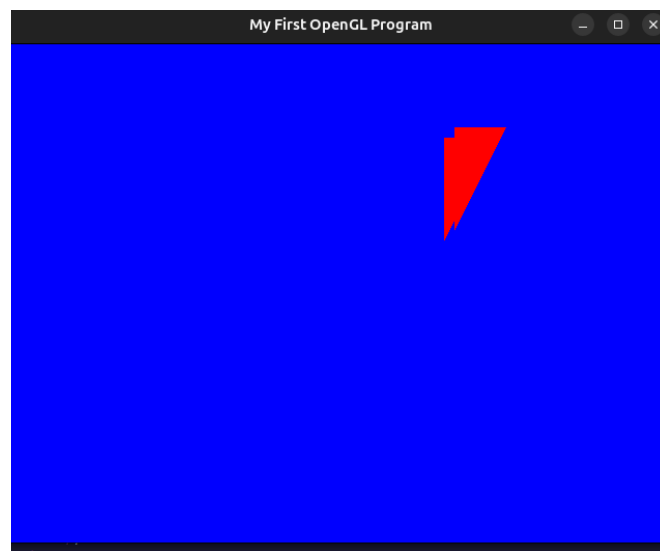


Figure 2: Αλλάξαμε το background από χρώμα άσπρο το κάναμε μπλε. Το πετύχαμε αυτό δίνοντας διαφορετικά ορίσματα στην `glClearColor`.

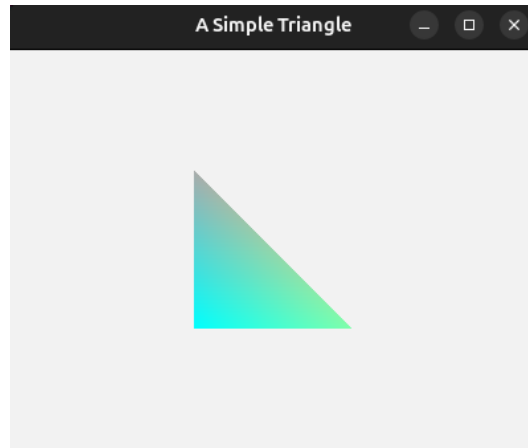


Figure 3: Αλλάξαμε τα χρώματα από τα τρία σημεία τα οποία σχηματίζουν το τρίγωνο. Το πετύχαμε αυτό δίνοντας διαφορετικά ορίσματα στην `glColor3f`.

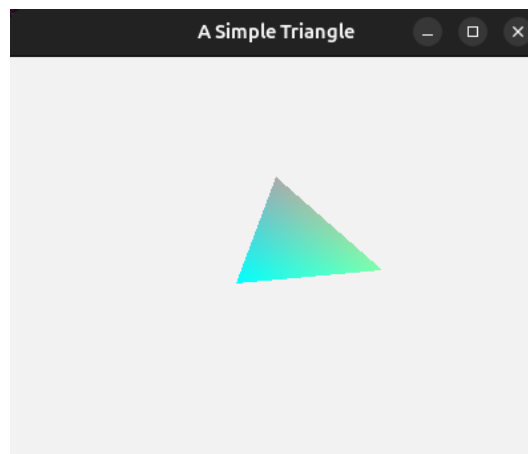


Figure 4: Αλλάξαμε τα σημεία τα οποία σχηματίζουν το τρίγωνο. Το πετύχαμε αυτό δίνοντας διαφορετικά σημεία στην `glVertex2f`.

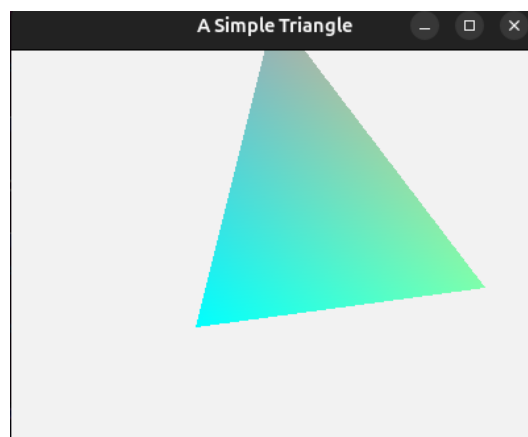
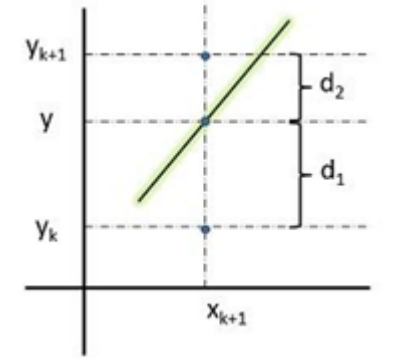


Figure 5: Αλλάξαμε την περιοχή προβολής, την μικρώναμε. Το πετύχαμε αυτό ορίζοντας μικρότερο όριο συντεταγμένων στην `gluOrtho2d`.

Exercise 2

2.1 Algorithm Derivation

Για τον σχεδιασμό μιας γραμμής με τον αλγόριθμο Bresenham, η κλίση της ευθείας m πρέπει να είναι στο πρώτο οκτατημόριο, δηλαδή $0 < m < 1$. Σε αυτή την περίπτωση, η μεταβλητή που αντιπροσωπεύει την μετατόπιση στον άξονα x αυξάνεται συνεχώς. Αντίθετα, για τον άξονα y χρειάζεται μια συνθήκη αύξησης, καθώς οι τιμές του x δεν αντιστοιχούν μία προς μία στις τιμές του y .



Σχήμα 6: Υπολογισμός αποστάσεων

Σχηματικά η συνθήκη αυτή επιλέγει το κοντινότερο pixel στην θεωρητική τιμή της συνάρτησης την ευθείας για ένα συγκεκριμένο x , όπως φαίνεται στο σχήμα. Για την επιτευξη αυτού, θα χρησιμοποιήσουμε τις αποστάσεις των σημείων $P_0(x_k, y_k)$, $P_1(x_k, y_{k+1})$ από το σημείο $P_{line}(x_k, y_{line})$ όπου y_{line} είναι η πραγματική τιμή της ευθείας στο σημείο αυτό, και τις ονομάζουμε d_1, d_2 αντίστοιχα. Συνεπώς έχουμε την παρακάτω μαθηματική ανάπτυξη:

$$d_1 = y_{line} - y_k = m \cdot (x_k + 1) + c - y_k$$

$$d_2 = y_k + 1 - y_{line} = y_k + m \cdot (x_k + 1) - c$$

Χρησιμοποιώντας τις παραπάνω σχέσεις και την σχηματική αναπαράσταση της λύσης, μπορούμε να αποφανθούμε ότι το $sign(d_k)$, $d_k = d_1 - d_2$ αποτελεί την συνθήκη αύξησης της μεταβλητής μετατόπισης του y . Για την διαφορά αυτή έχουμε:

$$d_1 - d_2 = (m \cdot (x_k + 1) + c - y_k) - (y_k + m \cdot (x_k + 1) - c) \Rightarrow$$

$$d_1 - d_2 = (2m \cdot (x_k + 1) - 2y_k + 2c - 1) \xrightarrow{\cdot \Delta x, m = \frac{\Delta y}{\Delta x}}$$

$$P_k = \Delta x \cdot (d_1 - d_2) = 2\Delta y \cdot (x_k + 1) - 2\Delta y \cdot y_k + \underbrace{2\Delta y + 2\Delta x \cdot c - \Delta x}_{\text{σταθερό}}$$

Παρατηρούμε ότι το επισημασμένο κομμάτι την εξίσωσης παραμένει σταθερό όσο αυξάνουμε το k :

$$P_k = 2\Delta y \cdot (x_k + 1) - 2\Delta y \cdot y_k + c_1$$

$$P_{k+1} = 2\Delta y \cdot (x_{k+1} + 1) - 2\Delta y \cdot y_{next} + c_1, \quad y_{next} = \begin{cases} y_{next} = y_k + 1, & \text{if } d_k \geq 0 \\ y_{next} = y_k, & \text{if } d_k < 0 \end{cases}$$

Από τις παραπάνω εξισώσεις προκύπτει:

$$P_{k+1} - P_k = 2\Delta y \cdot (x_{k+1} - x_k) - 2\Delta y \cdot (y_{next} - y_k) + \underbrace{c_1 - c_1}_{\text{ακυρώνεται}}$$

Επειδή ισχύει $x_{k+1} = x_k + 1$ και $y_{next} = y_k + 1$ ή $y_{next} = y_k$, έχουμε:

$$\text{Αν } y_{next} - y_k = 0: P_{k+1} - P_k = 2\Delta y > 0 \quad (0)$$

$$\text{Av } y_{next} - y_k = 1: P_{k+1} - P_k = 2\Delta y - 2\Delta x \xrightarrow{0 < \frac{\Delta y}{\Delta x} < 1} P_{k+1} - P_k < 0 \quad (1)$$

$$\text{Av } (0) \Rightarrow y_{next} = y_k :$$

$$P_{k+1} = P_k + 2\Delta y \cdot (x_k + 1) - 2\Delta x \cdot (y_k - y_k)$$

$$P_{k+1} = P_k + 2\Delta y$$

$$\text{Av } (1) \Rightarrow y_{next} = y_k + 1 :$$

$$P_{k+1} = P_k + 2\Delta y \cdot (x_k + 1) - 2\Delta x \cdot (y_k + 1 - y_k)$$

$$P_{k+1} = P_k + 2\Delta y - 2\Delta x$$

Συνεπώς :

$$P_{k+1} = \begin{cases} P_k + 2\Delta y - 2\Delta x, & \text{αν } y_{next} = y_k + 1 \\ P_k + 2\Delta y, & \text{αν } y_{next} = y_k \end{cases}$$

Απο πλευράς κώδικα με την απαλοιφή της floating m μεταβλητής μπορούμε να χρησιμοποιήσουμε μόνο integer μεταβλητές. Σε ψευδοκώδικα αυτό μοιάζει κάπως έτσι :

Algorithm 1 Bresenham Line Drawing Algorithm

```

1:  $x_0 \leftarrow x_L$ 
2:  $y_0 \leftarrow y_L$ 
3:  $dx \leftarrow x_H - x_L$ 
4:  $dy \leftarrow y_H - y_L$ 
5: if not ( $dx$  and  $dy$ ) then
6:   draw( $x_0, y_0$ )
7:   return
8: end if
9:  $p \leftarrow 2 dy - dx$ 
10: draw( $x_0, y_0$ )
11: while  $x_k < x_H$  do
12:    $x_k \leftarrow x_k + 1$ 
13:   if  $p \geq 0$  then
14:      $p \leftarrow p - 2 dx$ 
15:      $y_k \leftarrow y_k + 1$ 
16:   end if
17:    $p \leftarrow p + 2 dy$ 
18:   draw( $x_k, y_k$ )
19: end while
```

Για την υλοποίηση του γενικού αλγορίθμου του Bresenham, θα προσαρμόσουμε τον παραπάνω κώδικα ώστε να καλύπτει τα διαφορετικά οκτατημόρια, αξιοποιώντας τα χαρακτηριστικά μοτίβα που εμφανίζονται σε κάθε περίπτωση. Συγκεκριμένα, παρατηρούμε ότι οι μεταβολές των μεταβλητών μετατόπισης στους άξονες δεν αυξάνονται απαραίτητα κατά ένα, σε ορισμένες περιπτώσεις μάλιστα, μειώνονται. Επιπλέον, για $m > 1$ στο πρώτο τεταρτημόριο, οι ρόλοι των αξόνων x και y αντιστρέφονται, με τη μεταβλητή y να αναλαμβάνει πλέον τον ρόλο του οδηγού.

Από αυτή την οπτική, μπορούμε να γενικεύσουμε τον αλγόριθμο εισάγοντας μια πιο αφηρημένη έννοια για τις μεταβλητές μετατόπισης, τις leading και trailing. Επιπλέον, ανάλογα με την κατεύθυνση της ευθείας και τα άκρα της, μπορούμε να υπολογίσουμε τη σταθερά μετατόπισης που θα χρησιμοποιηθεί για τη μετακίνηση του σημείου σχεδίασης στους άξονες x και y .

Algorithm 2 Ο γενικός αλγόριθμος Bresenham

```
1:  $dstX \leftarrow |endPoint_x - startPoint_x|$ 
2:  $dstY \leftarrow |endPoint_y - startPoint_y|$ 
3: if not ( $dstX$  or  $dstY$ ) then
4:   drawPoint( $startPoint_x$ ,  $startPoint_y$ ,  $startPoint_{color}$ , 1)
5:   return
6: end if
7:  $condSlope \leftarrow (dstX \geq dstY)$ 
8:  $lastPixelIndex \leftarrow \max(dstX, dstY)$ 
9:  $\Delta color \leftarrow \frac{endPoint_{color} - startPoint_{color}}{lastPixelIndex}$ 
10:  $color \leftarrow startPoint_{color}$ 
11:
12: if  $condSlope$  then
13:    $leadingAxis \leftarrow startPoint_x$ 
14:    $trailingAxis \leftarrow startPoint_y$ 
15:    $endPointCoord \leftarrow endPoint_x$ 
16:    $signLeadingAxis \leftarrow \text{sign}(endPoint_x - startPoint_x)$ 
17:    $signTrailingAxis \leftarrow \text{sign}(endPoint_y - startPoint_y)$ 
18:    $dstLeadingAxis \leftarrow dstX$ 
19:    $dstTrailingAxis \leftarrow dstY$ 
20: else
21:    $leadingAxis \leftarrow startPoint_y$ 
22:    $trailingAxis \leftarrow startPoint_x$ 
23:    $endPointCoord \leftarrow endPoint_y$ 
24:    $signLeadingAxis \leftarrow \text{sign}(endPoint_y - startPoint_y)$ 
25:    $signTrailingAxis \leftarrow \text{sign}(endPoint_x - startPoint_x)$ 
26:    $dstLeadingAxis \leftarrow dstY$ 
27:    $dstTrailingAxis \leftarrow dstX$ 
28: end if
29:
30:  $error \leftarrow (2 \cdot dstTrailingAxis) - dstLeadingAxis$ 
31: for  $i \leftarrow 0$  to  $lastPixelIndex - 1$  do
32:   if  $condSlope$  then
33:     drawPoint( $leadingAxis$ ,  $trailingAxis$ ,  $color$ )
34:   else
35:     drawPoint( $trailingAxis$ ,  $leadingAxis$ ,  $color$ )
36:   end if
37:
38:    $leadingAxis \leftarrow leadingAxis + signLeadingAxis$ 
39:
40:   if  $error \geq 0$  then
41:      $error \leftarrow error - 2 \cdot dstLeadingAxis$ 
42:      $trailingAxis \leftarrow trailingAxis + signTrailingAxis$ 
43:   end if
44:    $error \leftarrow error + 2 \cdot dstTrailingAxis$ 
45:    $color \leftarrow color + \Delta color$ 
46: end for
```

2.2 Color Interpolation Extension

Για την υλοποίηση της παρεμβολής του χρώματος, εφόσον υποθέτουμε ομαλή μετάβαση χρώματος μπορούμε να χρησιμοποιήσουμε γραμμική παρεμβολή. Έστω σημεία P_0 και P_{n-1} με χρώματα $c_0 = (R_0, G_0, B_0)$ και $c_{n-1} = (R_{n-1}, G_{n-1}, B_{n-1})$ αντίστοιχα. Θα εφαρμόσουμε την παρεμβολή σε κάθε κανάλι του χρώματος ξεχωριστά. Εστιάζοντας στο ένα από τα 3 κανάλια βρίσκουμε το βήμα με το οποίο θα πρέπει να μεταβάλλεται το κόκκινο ώστε για το P_0 να είναι R_0 και για P_{n-1} να είναι R_{n-1} . Έστω P_i το i -ιοστό pixel που χρωματίζουμε με τιμή R_i για την οποία ισχύει:

$$R_i = R_0 + \frac{i}{n}(R_1 - R_0)$$

όπου n το πλήθος των συνολικών pixel που θα χρωματιστούν. Έπειτα για να βρούμε το βήμα που προστείνεται σε κάθε επανάληψη στο χρώμα θα βρούμε την διαφορά $R_{i+1} - R_i$:

$$\begin{aligned} R_{i+1} - R_i &= R_0 + \frac{i+1}{n-1}(R_1 - R_0) - (R_0 + \frac{i}{n-1}(R_1 - R_0)) \implies \\ R_{i+1} - R_i &= \cancel{R_0} + \frac{i}{\cancel{n-1}}(\cancel{R_1 - R_0}) + \frac{R_1 - R_0}{n-1} - \cancel{R_0} - \frac{i}{\cancel{n-1}}(\cancel{R_1 - R_0}) \implies \\ R_{i+1} - R_i &= \frac{R_1 - R_0}{n-1} \end{aligned}$$

Επομένως θα πρέπει σε κάθε επανάληψη να προσθέτουμε στην τιμή του χρώματος, $\text{delta} = \frac{R_1 - R_0}{n-1}$. Αντίστοιχα θα πρέπει να κάνουμε και για τα άλλα 2 κανάλια.

2.3 Example Test Case

Αρχικά βρίσκουμε τις απόλυτες τιμές των αποστάσεων dx, dy , οι οποίες είναι 10, 7 αντίστοιχα. Διαπιστώνουμε ότι η κλίση της ευθείας είναι μεταξύ $0 < m < 1$ άρα η μεταβλητή μετατόπισης που θα αυξάνεται συνεχώς είναι η x . Επιπλέον από τις απόστασεις dx, dy με βάση το πρόσημο τους υπολογίζουμε ότι το x θα μειώνεται, ενώ το y θα αυξάνεται. Το αρχικό σφάλμα υπολογίζεται από τον τύπο $\text{error} = 2 \cdot dy - 2 \cdot dx = 14 - 10 = 4$.

1η Επανάληψη: Το x μειώνεται και η νέα του τιμή είναι 7. Το error είναι θετικό άρα το y αυξάνεται και η νέα του τιμή είναι -3 . Η τελική τιμή του error είναι $\text{error} = -16 + 14 = -2$.

2η Επανάληψη: Το x μειώνεται και η νέα του τιμή είναι 6. Το error είναι αρνητικό άρα το y παραμένει σταθερό. Η τελική τιμή του error είναι $\text{error} = -2 + 14 = 12$.

3η Επανάληψη: Το x μειώνεται και η νέα του τιμή είναι 5. Το error είναι θετικό άρα το y αυξάνεται και η νέα του τιμή είναι -2 . Η τελική τιμή του error είναι $\text{error} = -8 + 14 = 6$.

4η Επανάληψη: Το x μειώνεται και η νέα του τιμή είναι 4. Το error είναι θετικό άρα το y αυξάνεται και η νέα του τιμή είναι -1 . Η τελική τιμή του error είναι $\text{error} = -14 + 14 = 0$.

5η Επανάληψη: Το x μειώνεται και η νέα του τιμή είναι 3. Το error είναι θετικό άρα το y αυξάνεται και η νέα του τιμή είναι 0. Η τελική τιμή του error είναι $\text{error} = -20 + 14 = -6$.

6η Επανάληψη: Το x μειώνεται και η νέα του τιμή είναι 2. Το error είναι αρνητικό άρα το y παραμένει σταθερό. Η τελική τιμή του error είναι $\text{error} = -6 + 14 = 8$.

7η Επανάληψη: Το x μειώνεται και η νέα του τιμή είναι 1. Το error είναι θετικό άρα το y αυξάνεται και η νέα του τιμή είναι 1. Η τελική τιμή του error είναι $\text{error} = -12 + 14 = 2$.

8η Επανάληψη: Το x μειώνεται και η νέα του τιμή είναι 0. Το error είναι θετικό άρα το y αυξάνεται και η νέα του τιμή είναι 2. Η τελική τιμή του error είναι $\text{error} = -18 + 14 = -4$.

9^η Επανάληψη: Το x μειώνεται και η νέα του τιμή είναι -2 . Το $error$ είναι αρνητικό άρα το y παραμένει σταθερό. Η τελική τιμή του $error$ είναι $error = -4 + 14 = 10$.

Table 1: Οι τιμές x, y και του σφάλματος σε κάθε βήμα του αλγορίθμου.

Βήματα	x	y	Σφάλμα	(x,y)
0	8	-4	4	(8,-4)
1	7	-3	-2	(7,-3)
2	6	-3	12	(6,-3)
3	5	-2	6	(5,-2)
4	4	-1	0	(4,-1)
5	3	0	-6	(3,0)
6	2	0	8	(2,0)
7	1	1	2	(1,1)
8	0	2	-4	(0,2)
9	-1	2	10	(-1,2)
10	-2	3	τέλος	(-2,3)

Βήματα	R	G	B	Hex
0	255.0	0.0	0.0	#FF0000
1	229.5	25.5	0.0	#E51A00
2	204.0	51.0	0.0	#CC3300
3	178.5	76.5	0.0	#B34D00
4	153.0	102.0	0.0	#996600
5	127.5	127.5	0.0	#7F7F00
6	102.0	153.0	0.0	#669900
7	76.5	178.5	0.0	#4DB300
8	51.0	204.0	0.0	#33CC00
9	25.5	229.5	0.0	#1AE500
10	0.0	255.0	0.0	#00FF00

Table 2: Οι τιμές RGB σε κάθε βήμα του αλγορίθμου.

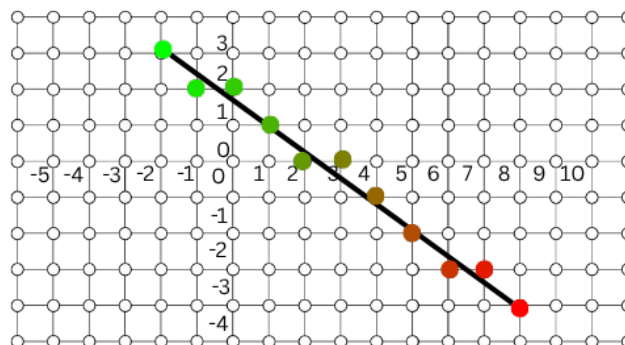


Figure 7: Η γραμμή με βάση τον αλγόριθμο του Bresenham.

Exercise 3

3.1 Algorithm Derivation:

Για την πραγματοποίηση του αλγορίθμου Bresenham για κύκλους που το κέντρο τους δεν είναι η αρχή των αξόνων, η μόνη διαφορά είναι ο τρόπος που ζωγραφίζουμε τα pixel. Όταν τα ζωγραφίζουμε προσθέτουμε και το offset δηλαδή τα σημεία x, y τα οποία αποτελούν το κέντρο του κύκλου. Επίσης όταν σχεδιάζουμε έναν κύκλο και ξέρουμε ένα σημείο πρακτικά γνωρίζουμε και άλλα 7 λόγω συμμετρίας, οπότε ο αλγόριθμος σχεδιάζει μόνο για ένα οκτατημόριο. Παρακάτω είναι ο ψευδοκώδικας του αλγορίθμου.

Algorithm 3 Ο γενικός αλγόριθμος του Bresenham για κύκλους

```
1:  $x_{center} \leftarrow 0$ 
2:  $y_{center} \leftarrow r$ 
3:  $x_{point} \leftarrow x_o$ 
4:  $y_{point} \leftarrow y_o$ 
5:  $error \leftarrow 3 - 2r$ 
6: while  $x_{center} \leq y_{center}$  do
7:   draw( $x_{point} + x_{center}, y_{point} + y_{center}$ )    (x, y)
8:   draw( $x_{point} + y_{center}, y_{point} + x_{center}$ )    (y, x)
9:   draw( $x_{point} - x_{center}, y_{point} + y_{center}$ )    (-x, y)
10:  draw( $x_{point} + y_{center}, y_{point} - x_{center}$ )    (y, -x)
11:  draw( $x_{point} - x_{center}, y_{point} - y_{center}$ )    (-x, -y)
12:  draw( $x_{point} - y_{center}, y_{point} - x_{center}$ )    (-y, -x)
13:  draw( $x_{point} - y_{center}, y_{point} + x_{center}$ )    (-y, x)
14:  draw( $x_{point} + x_{center}, y_{point} - y_{center}$ )    (x, -y)
15:   $x_{center} \leftarrow x_{center} + 1$ 
16:  if  $error \geq 0$  then
17:     $y_{center} \leftarrow y_{center} - 1$ 
18:     $error \leftarrow error - 4 \times y_{center}$ 
19:  end if
20:   $error \leftarrow error + 4 \times x_{center} + 2$ 
21: end while
```

3.2 Example Test Case:

Αρχικοποιούμε τις τιμές x, y με 0, 7 αντίστοιχα. Η αρχική τιμή του σφάλματος είναι $error = 3 - 2 \cdot r = 3 - 14 = -11$.

1η Επανάληψη: Το x αυξάνεται και η νέα του τιμή είναι 1. Το $error$ είναι αρνητικό άρα το y παραμένει σταθερό. Η τελική τιμή του $error$ είναι $error = -11 + 4 + 2 = -5$.

2η Επανάληψη: Το x αυξάνεται και η νέα του τιμή είναι 2. Το $error$ είναι αρνητικό άρα το y παραμένει σταθερό. Η τελική τιμή του $error$ είναι $error = -5 + 4 \cdot 2 + 2 = 5$.

3η Επανάληψη: Το x αυξάνεται και η νέα του τιμή είναι 3. Το $error$ είναι θετικό άρα το y αυξάνεται και η νέα του τιμή είναι 6. Η τελική τιμή του $error$ είναι $error = -19 + 4 \cdot 3 + 2 = -5$.

4η Επανάληψη: Το x αυξάνεται και η νέα του τιμή είναι 4. Το $error$ είναι αρνητικό άρα το y παραμένει σταθερό. Η τελική τιμή του $error$ είναι $error = -5 + 4 \cdot 4 + 2 = 13$.

5η Επανάληψη: Το x αυξάνεται και η νέα του τιμή είναι 5. Το $error$ είναι θετικό άρα το y αυξάνεται και η νέα του τιμή είναι 5. Η τελική τιμή του $error$ είναι $error = -7 + 4 \cdot 5 + 2 = 15$.

Εδώ τελειώνει ο αλγόριθμος γιατί το x ξεπέρασε το y . Ο αλγόριθμος μας υπολογίζει τα σημεία για το $\frac{1}{8}$ του κύκλου, όμως λόγω συμμετρίας όταν γνωρίζουμε ένα σημείο του κύκλου ξέρουμε και τα άλλα 7. Παρακάτω μπορούμε να δούμε τον πίνακα όπου περιέχει όλα τα βήματα του αλγορίθμου.

Table 3: Οι τιμές x,y και του σφάλματος σε κάθε βήμα του αλγορίθμου.

x	y	Σφάλμα	(x,y)
0	7	-11	(3,12)
1	7	-5	(4,12)
2	7	5	(5,12)
3	6	-5	(6,11)
4	6	13	(7,11)
5	5	15	(8,10)

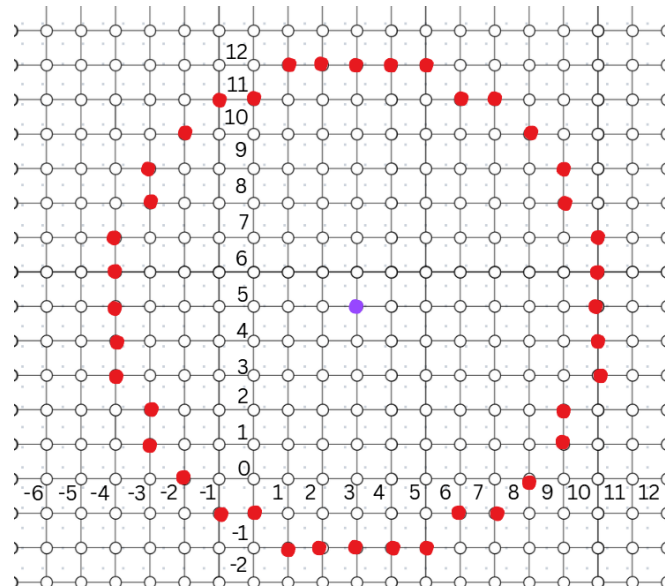


Figure 8: Ο κύκλος με βάση τον αλγόριθμο του Bresenham.

Exercise 4

4.1 Implementation structure

Η υλοποίηση μας ορίζει 3 structs για την περιγραφή των σημείων, ευθειών και χρωμάτων. Επιπλέον ορίζει ένα enum για την επιλογή της λειτουργίας αντιταύτισης (βλέπε 2). Ορίζει συναρτήσεις για την σχεδίαση σημείων και ευθειών βασισμένη στον αλγόριθμο της άσκησης 2. Επίσης υλοποιεί και τον αλγόριθμο του Xiaolin Wu σε ξεχωριστή συνάρτηση σχεδίασης ευθείας και υπάρχει η επιλογή εναλλαγή μεταξύ των δυο συναρτήσεων. Οι ευθείες που δημιουργούνται από τον χρήστη απόθηκεύονται σε πίνακα καθώς στο πρόγραμμα μας υπάρχει η επιλογή της αλλαγής μεγέθους του παραθύρου όποτε απαιτείται επανασχεδιασμός των ευθειών. Η συγκεκριμένη υλοποίηση επιτρέπει την εύκολη επέκταση του προγράμματος για την προσθήκη περισσότερων σχημάτων το οποίο και υλοποιούμε στην άσκηση 5.

4.2 Integration of the algorithm

Η υλοποίηση μας βασίζεται στην έκδοση του αλγορίθμου που δημιουργήσαμε στην άσκηση 2. Επιπλέον υλοποιεί μιας μορφής αντιταύτισης χρησιμοποιώντας την παράμετρο απόφασης του αλγορίθμου για την οποία ισχύει $P = \Delta x \cdot (d_2 - d_1) \implies d_2 - d_1 = \frac{P}{\Delta x}$, ώστε να βρει την φωτεινότητα των δυο γειτονικών pixel. Η διαφορά των αποστάσεων μεταφράζεται σε ποσοστό φωτεινότητας για το αντίστοιχο pixel.

4.3 User handling and color handling

Στον χρήστη δίνεται η επιλογή με την χρήση input από το πληκτρολόγιο να διαλέξει το χρώμα των ευθειών αλλά και τη μέθοδο αντιταύτισης. Συγκεκριμένα, με το πλήκτρο **a** επιλέγεται η μέθοδος αντιταύτισης για την επόμενη ευθεία. Η αρχική επιλογή είναι να μην εκτελεί αντιταύτιση, ενώ πατώντας το **a** εναλλάσσεται κυκλικά μεταξύ των εξής μεθόδων: **NONE**, **BRESENHAM**, **WU**. Με τα πλήκτρα **r**, **g**, **b** θέτει στο αντίστοιχο χρώμα (κόκκινο, πράσινο, μπλε) για το επόμενο σημείο που θα επιλεγεί. Σε περίπτωση σχεδίασης γραμμής αν έχουν επιλεγεί διαφορετικά χρώματα για το αρχικό και τελικό σημείο τότε πραγματοποιείται παρεμβολή για να έχουμε ομαλή μετάβαση του χρώματος από το αρχικό στο τελικό. Το χρώμα του κάθε pixel όπως και η μέθοδος αντιταύτισης αποθηκεύονται στο **struct Point** και **struct Line** και μπορούν να χρησιμοποιηθούν για τον επανασχεδιασμό τους εαν χρειαστεί λόγω αλλαγής μεγέθους του παραθύρου.

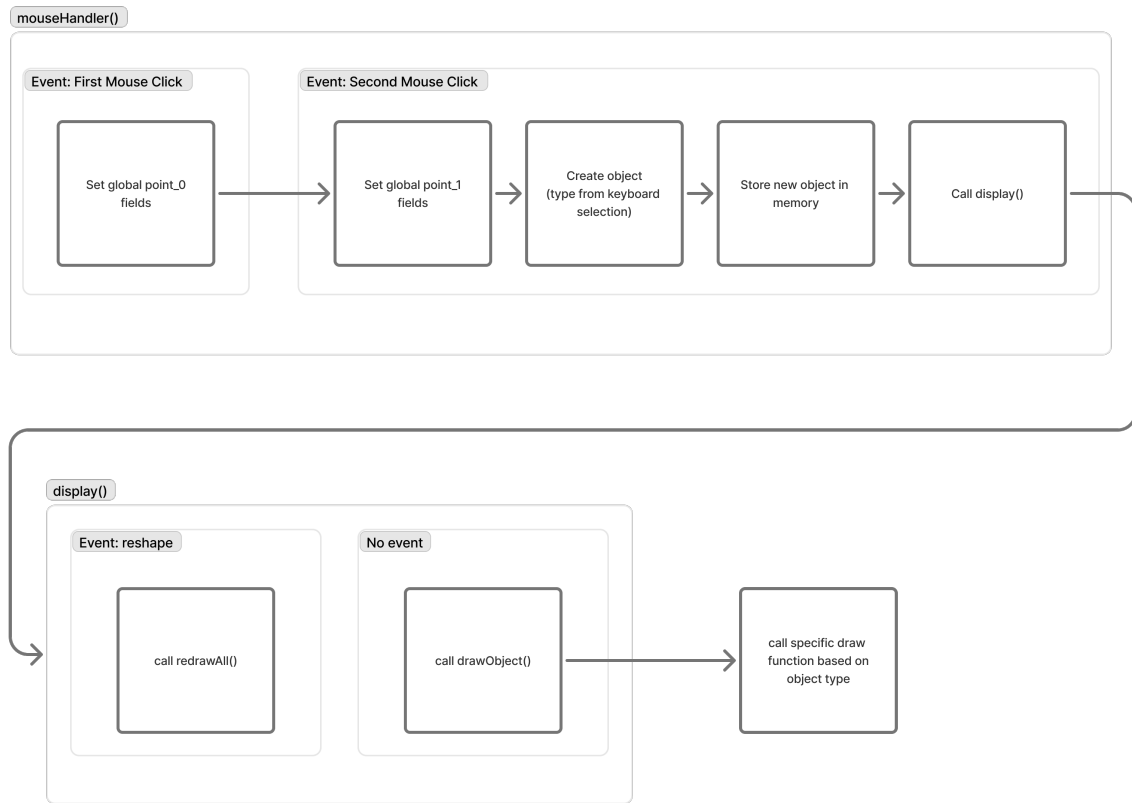
4.4 Summary

- Πλήκτρο 'a': Εναλλάσσει κυκλικά την μέθοδο αντιταύτισης μεταξύ των **NONE**, **BRESENHAM**, **WU**.
- Πλήκτρο 'r': Θέτει το χρώμα για το επόμενο σημείο κόκκινο.
- Πλήκτρο 'g': Θέτει το χρώμα για το επόμενο σημείο πράσινο.
- Πλήκτρο 'b': Θέτει το χρώμα για το επόμενο σημείο μπλε.
- Αριστερό click: Επιλέγει σημείο και κάθε 2 σημεία σχεδιάζει μια ευθεία με την επιλεγμένη μέθοδο αντιταύτισης και τα επιλεγμένα χρώματα στο αρχικό και τελικό σημείο.
- Δεξί click: Καθαρίζει το παράθυρο.

Exercise 5

5.1 Implementation structure and algorithm integration

Η λογική του προγράμματος περιγράφεται στο παρακάτω γράφημα, ουσιαστικά φαίνεται η διαδικασία που ακολουθεί το πρόγραμμα από την είσοδο του χρήστη μέχρι τον σχεδιασμό ενός Object.



Σχήμα 9: Ροή προγράμματος

Η υλοποίηση μας επιτρέπει τον σχεδιασμό πολλαπλών object και επιπλέον επιτρέπει την επαναπροσαρμογή μεγέθους του παραθύρου. Η υλοποίηση περιλαμβάνει τα παρακάτω είδους objects (βλέπε 4) και συναρτήσεις διαχείρισης αυτών :

```
1 void storeObj(Object object);
2 Object newObj(ObjectType type);
3 void drawObject(Object object);
```

Listing 1: Management functions

- Όπου η storeObj αποθηκεύει ένα αντικείμενο στην μνήμη η οποία χρησιμοποιείται για την επανασχεδίαση των αντικειμένων σε περίπτωση επαναπροσαρμογή μεγέθους του παραθύρου.
- Η συνάρτηση newObj δημιουργεί ένα αντικείμενο του οποίου ο τύπος ορίζεται από την μεταβλητή typeSelector με την βοήθεια του πληκτρολογίου (αναφέρεται παρακάτω) και των στοιχείων από τα σημεία που έχει ορίσει ο χρήστης.
- Τέλος η drawObject σχεδιάζει ένα αντικείμενο στην οθόνη χρησιμοποιώντας διαφορετικά draw methods ανάλογα το είδος του αντικειμένου.

Όσο για τις μεθόδους σχεδίασης των αντικειμένων αξίζει να αναφερθεί η `drawCircleFilled`, καθώς οι άλλες προαναφέρθηκαν στο Exercise4, η οποία χρησιμοποιεί σχεδόν τον ίδιο αλγόριθμο με τον απλό κύκλο. Η υλοποίηση αυτή διαφέρει λόγω της ομαδοποίησης των σημείων σε οριζόντιες ευθείες οι οποίες ξεκινάνε από τα οκτατημόρια που βρίσκονται αριστερά από τον άξονα y και καταλήγουν σε αυτά δεξιά του (βλέπε παράδειγμα 10).

5.2 User handling and color handling

Με την χρήση του πληκτρολογίου μπορούμε να προβούμε στις ίδιες ενέργειες με την άσκηση 4 και ακόμα :

- Επιλογή 'P' επιτρέπει τον σχεδιασμό ενός σημείου, (το δεύτερο click στην περίπτωση αυτή αποτελεί το confirm).
- Επιλογή 'C', που είναι και η προεπιλογή στην αρχή του προγράμματος, και επιτρέπει τον σχεδιασμό ενός κύκλου.
- Επιλογή 'F' που επιτρέπει τον σχεδιασμό ενός 'γεμισμένου κύκλου'.
- Τέλος υποστηρίζεται ο σχεδιασμός color interpolated γεμισμένου κύκλου με την χρήση διαφορετικών χρωμάτων κατά την επιλογή των σημείων του.



Σχήμα 10: Κομμάτι γεμισμένου κύκλου με ευθείες από το 4ο στο 1ο οκτατημόριο

Specifications

Για να εκτελέσουμε και να ελέγξουμε τις υλοποιήσεις μας χρησιμοποιήσαμε το ακόλουθο σύστημα:

- CPU: AMD Ryzen 7 5800X 8-Core Processor
- Caches (sum of all):
 - L1d: 256 KiB (8 instances)
 - L1i: 256 KiB (8 instances)
 - L2: 4 MiB (8 instances)
 - L3: 32 MiB (1 instance)
- Memory: 32GB DDR4
- GPU: GeForce RTX 3060 Lite Hash Rate
- Compiler version: gcc (Ubuntu 13.3.0-6ubuntu2 24.04) 13.3.0
- Operating System: Ubuntu LTS 24.04

Appendix

Code of Exercise 4

Here is our program for exercise 4. File name: exercise4.cpp

```
1 typedef enum {
2     NONE,
3     BRESENHAM,
4     WU
5 } AntialiasingMode;
6
7 typedef struct {
8     float red, green, blue;
9 } RGB;
10
11 typedef struct {
12     int x, y;
13     RGB rgb;
14 } Point;
15
16 typedef struct {
17     Point start;
18     Point end;
19     AntialiasingMode mode;
20 } Line;
21
22 void drawLine(Line line) {
23     int error = 0;
24     int leadingAxis = 0, trailingAxis = 0, endPointCoord = 0;
25     int signLeadingAxis = 0, signTrailingAxis = 0,
26         dstLeadingAxis = 0, dstTrailingAxis = 0;
27     float alpha, mainPixel, secPixel;
28
29     Point endPoint = line.end;
30     Point startPoint = line.start;
31
32     // Calculate axis distance
33     int dstX = abs(endPoint.x - startPoint.x);
34     int dstY = abs(endPoint.y - startPoint.y);
35
36     // Set default color
37     glColor3f(startPoint.rgb.red, startPoint.rgb.green, startPoint.rgb.blue);
38
39     // Starting point == End point
40     if (!(dstX || dstY)) {
41         drawPoint(startPoint.x, startPoint.y);
42         return;
43     }
44
45     int condSlope = dstX >= dstY;
46     int lastPixelIndex = max(dstX, dstY);
47
48     RGB delta = {
49         (line.end.rgb.red - line.start.rgb.red) / lastPixelIndex,
50         (line.end.rgb.green - line.start.rgb.green) / lastPixelIndex,
51         (line.end.rgb.blue - line.start.rgb.blue) / lastPixelIndex
52     };
53
54     // Pick leading and trailing axis
55     if (condSlope) {
56         // Initialise points
```

```

57     leadingAxis = startPoint.x;
58     trailingAxis = startPoint.y;
59     endPointCoord = endPoint.x;
60     // Initialise axis variables
61     signLeadingAxis = sign(endPoint.x - startPoint.x);
62     signTrailingAxis = sign(endPoint.y - startPoint.y);
63     dstLeadingAxis = dstX;
64     dstTrailingAxis = dstY;
65 } else {
66     // Initialise points
67     leadingAxis = startPoint.y;
68     trailingAxis = startPoint.x;
69     endPointCoord = endPoint.y;
70     // Initialise axis variables
71     signLeadingAxis = sign(endPoint.y - startPoint.y);
72     signTrailingAxis = sign(endPoint.x - startPoint.x);
73     dstLeadingAxis = dstY;
74     dstTrailingAxis = dstX;
75 }
76
77 glBegin(GL_POINTS);
78 RGB color = startPoint.rgb; // use local color copy
79
80 // Initialise error variable
81 error = (2 * dstTrailingAxis) - dstLeadingAxis;
82
83 //? Update draw buffer every line instead of pixel
84 //? This is faster and per pixel makes 0 sense
85 for (int i = 0; i <= lastPixelIndex; ++i) { // loop over exact number of
86     pixels
87     // Compute fractional coverage from error
88     alpha = fabs((float)error) / (2.0f * (float)dstLeadingAxis);
89     mainPixel = alpha > 0.5 ? alpha : 1 - alpha;
90     secPixel = 1 - mainPixel;
91
92     // Draw main pixel
93     glColor4f(startPoint.rgb.red, startPoint.rgb.green, startPoint.rgb.
94     blue, line.mode == BRESENHAM ? mainPixel : 1);
95
96     if (condSlope)
97         glVertex2i(leadingAxis, trailingAxis);
98     else
99         glVertex2i(trailingAxis, leadingAxis);
100
101     if (line.mode == BRESENHAM) {
102         // Draw neighbor pixel (one step toward the slope direction)
103         glColor4f(startPoint.rgb.red, startPoint.rgb.green, startPoint.rgb
104         .blue, secPixel);
105
106         if (condSlope)
107             glVertex2i(leadingAxis, trailingAxis + signTrailingAxis);
108         else
109             glVertex2i(trailingAxis + signTrailingAxis, leadingAxis);
110     }
111
112     leadingAxis += signLeadingAxis;
113     if (error >= 0) {
114         error -= 2 * dstLeadingAxis;
115         trailingAxis += signTrailingAxis;
116     }
117     error += 2 * dstTrailingAxis;

```

```

116     startPoint.rgb.red += delta.red;
117     startPoint.rgb.green += delta.green;
118     startPoint.rgb.blue += delta.blue;
119
120 }
121 glEnd();
122 }

```

Listing 2: drawLine function and defined structs of exercise 4.

Code of Exercise 5

Here is our program for exercise 5. File name: exercise5.cpp

```

1  void drawCircle(Circle circle) {
2      Point centerPoint = circle.center;
3      int radius = circle.radius;
4      int xOffset = 0, yOffset = radius;
5      int error = 3 - (2 * radius);
6
7      glBegin(GL_POINTS);
8      glColor3f(centerPoint.rgb.red, centerPoint.rgb.green, centerPoint.rgb.blue
9  );
10     while(xOffset <= yOffset) {
11         int xPos = centerPoint.x + xOffset;
12         int xNeg = centerPoint.x - xOffset;
13         int yPos = centerPoint.y + yOffset;
14         int yNeg = centerPoint.y - yOffset;
15         int yxPos = centerPoint.x + yOffset;
16         int yxNeg = centerPoint.x - yOffset;
17         int xyPos = centerPoint.y + xOffset;
18         int xyNeg = centerPoint.y - xOffset;
19
20         glVertex2i(xPos, yPos);
21         glVertex2i(yxPos, xyPos);
22         glVertex2i(xNeg, yPos);
23         glVertex2i(yxPos, xyNeg);
24         glVertex2i(xNeg, yNeg);
25         glVertex2i(yxNeg, xyNeg);
26         glVertex2i(xPos, yNeg);
27         glVertex2i(yxNeg, xyPos);
28
29         xOffset++;
30         if (error >= 0) {
31             yOffset--;
32             error -= 4 * yOffset;
33         }
34         error += (4 * xOffset) + 2;
35     }
36     glEnd();
37     glFlush();
38 }

```

Listing 3: drawCircle function and defined structs from exercise 5


```

1 typedef enum {
2     NONE,
3     BRESENHAM,
4     WU
5 } AntialiasingMode;
6
7 typedef enum {
8     OBJ_POINT,
9     OBJ_LINE,
10    OBJ_CIRCLE,
11    OBJ_FCIRCLE
12 } ObjectType;
13
14 typedef struct {
15     float red, green, blue;
16 } RGB;
17
18 typedef struct {
19     int x, y;
20     RGB rgb;
21 } Point;
22
23 typedef struct {
24     Point start;
25     Point end;
26     AntialiasingMode mode;
27 } Line;
28
29 typedef struct {
30     Point center;
31     int radius;
32 } Circle;
33
34 typedef struct {
35     Point center;
36     RGB secondary;
37     int radius;
38 } FilledCircle;
39
40 typedef union {
41     Point point;
42     Line line;
43     Circle circle;
44     FilledCircle fcircle;
45 } ObjectData;
46
47 typedef struct {
48     ObjectType type;
49     ObjectData data;
50 } Object;

```

Listing 4: drawLine function of exercise 4.